

PetCare Scheduler: A UML-Driven Veterinary Appointment System

Name:

Raineer P. Tamiat

Course/Subject:

System Integration and Architecture 2

Instructor's Name:

Mr. Jeremiah C. Dela Cruz

May 21, 2026

I. Introduction

Background of the Project

Many veterinary clinics still rely on manual appointment books or simple messaging workflows to organize clinic visits. This reliance on decentralized administrative procedures creates operational bottlenecks, introduces systemic data risks, and hinders effective client relationship tracking. *PetCare Scheduler* was engineered to replace these legacy habits with a modern, structured web application environment.

Purpose of the System

The purpose of this project is to implement an automated web-based appointment scheduling platform. The platform allows pet owners to perform self-service booking tasks asynchronously while equipping veterinary professionals with a real-time clinical workspace to process incoming consultation flows smoothly.

Importance of the Project

Automating the scheduling ecosystem improves internal facility organization, cuts down clinic staff workloads, and optimizes practitioner calendar use. It replaces unreliable physical text logs with a stateful data architecture, creating a more consistent and professional experience for both pet owners and veterinary personnel.

Problems the System Aims to Solve

- **Double Bookings and Scheduling Overlaps:** Eradicates conflicting reservations via real-time schema validation constraints.
- **Fragmented and Lost Animal Records:** Prevents decoupled data entries by binding patient tracking tables together.

- **Stateless Dynamic Operations Failure:** Solves asynchronous API authentication drops (e.g., "Unauthenticated" response alerts during live triage pipeline adjustments) by forcing secure, shared-origin credential pass-throughs.
- **Manual Administrative Overhead:** Minimizes scheduling friction by introducing dynamic status adjustments directly inside the clinical grid.

II. Objectives of the System

General Objective

To design, develop, and document a secure web-based veterinary appointment scheduling system utilizing a UML-driven structural layout backed by a MySQL database infrastructure.

Specific Objectives

- To enable pet owners to book, view, and cancel clinic appointments independently via a secure customer-facing web dashboard.
- To provide veterinarians with an optimized workflow triage table to monitor clinical queues and manage schedules efficiently.
- To build a database schema that models system entities and maintains data relationships across users, animals, and consultation slots.
- To secure backend endpoints from unauthenticated request script drops by mapping asynchronous operations through web session middleware guards.

III. Scope and Limitations

System Capabilities and Included Features

- **User Authentication Engine:** Features credential verification and registration modules for both Pet Owners and Veterinarians.
- **Pet Profile Register:** Allows clients to store and update vital pet metrics (Name, Type, Breed, Age) inside the system database.
- **Appointment Tracker:** Provides interactive tools for submitting booking slots, processing cancellations, and monitoring upcoming clinical queues.
- **Clinical Triage Workspace:** Allows veterinarians to review pending booking entries, claim patient cases, or delete processed records.

System Limitations and Exclusions

- **Financial Payment Gateways:** Includes scheduling and workflow management but completely excludes external online digital payment modules.
- **Mobile Platform Adaptations:** Built strictly as a responsive web platform; standalone mobile application distributions (iOS or Android) are excluded.
- **Advanced Medical Diagnostics:** Operates as an administrative workspace pipeline and scheduling log; clinical diagnostic software modules or automated algorithmic health evaluations are not included.

IV. System Description

Overview of the System

The *PetCare Scheduler* application runs on an object-relational Model-View-Controller framework. The platform segments operations based on structural roles, providing client interfaces for data input and veterinary views for data verification. To manage real-time triage updates without forcing full-page reloads, the system uses asynchronous backend endpoints. These endpoints require matching session tokens to protect against data manipulation risks.

Major Modules and Features

- **Identity Management Module:** Manages registrations and secure login flows, assigning account roles to control feature permissions.
- **Pet Registry Record:** Houses pet descriptive profiles, keeping animals structurally linked to their registered owners.
- **Dynamic Clinic Pipeline:** A dashboard table tracking active consultation dates, service requirements, and states.
- **Asynchronous Execution Layer:** Uses JavaScript fetch interactions to process triage actions (like claiming a slot or purging records) cleanly in the background.

User Roles and Functionalities

1. **Pet Owner (Client):** Registers accounts, builds profiles for multiple animals, books consultations, tracks request states, and submits cancellation requests.
2. **Veterinarian:** Reviews incoming queues, audits clinical details, claims open assignments, updates scheduling states, and purges canceled logs from the active database.

V. Technologies Used

Programming Languages

- **PHP:** Used as the backend language to coordinate object relationships, validate incoming data payloads, and execute business logic.
- **JavaScript (ES6):** Controls user dashboard states, handles status adjustments, and runs background network requests using secure fetch parameters.
- **HTML5 & CSS3:** Generates the layout, design structures, and styling configurations for the application dashboards.

Frameworks and Libraries

- **Laravel Framework:** Provides the architectural environment, including authentication guards, Eloquent database model relationships, and defensive middleware stacks.
- **FontAwesome v6.0:** Supplies vector icons for dashboard sidebars and clinical triage badges.

Database Management System

- **MySQL:** Acts as the persistent relational database management system, housing data structures and enforcing foreign key constraints.

Development Tools Used

- **Visual Studio Code:** The integrated development environment utilized to write and debug application components.
- **Artisan CLI:** Laravel's integrated terminal interface used to run database migrations and reset route registration states.
- **Web Browser Development Inspection Monitors:** Used to evaluate network payload parameters, track cookies, and trace asynchronous JavaScript execution errors.

VI. System Design

Use Case Diagram

The Use Case Diagram maps out the access boundaries of the application, splitting scheduling controls on the client side from triage tracking features on the veterinary side.

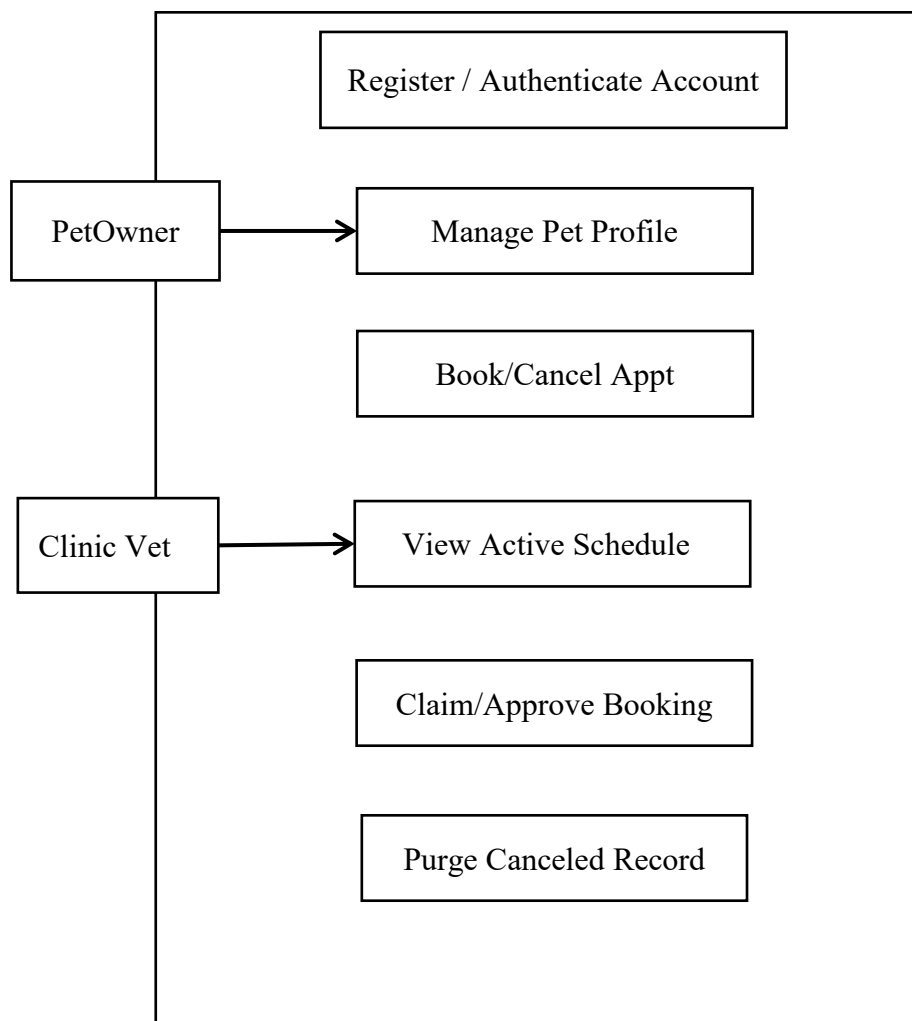


Figure 1: Use Case Diagram

System Flowchart

The flowchart outlines the execution path of the application. It verifies account types upon login and ensures that frontend update requests pass through middleware validation steps successfully.

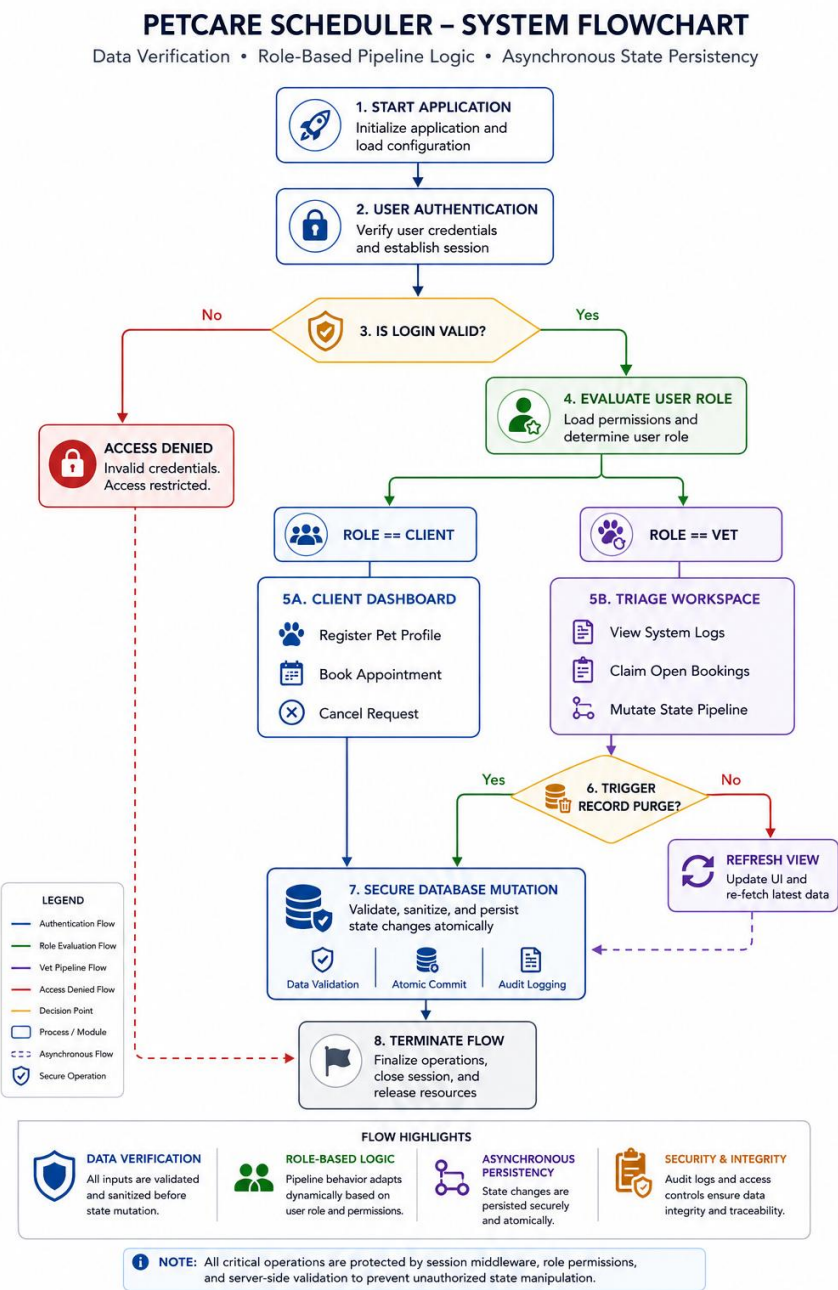


Figure 2: System Flowchart

System UML Class Diagram

The layout below illustrates the structural modeling of the application, establishing entity properties and detailing data associations across models.

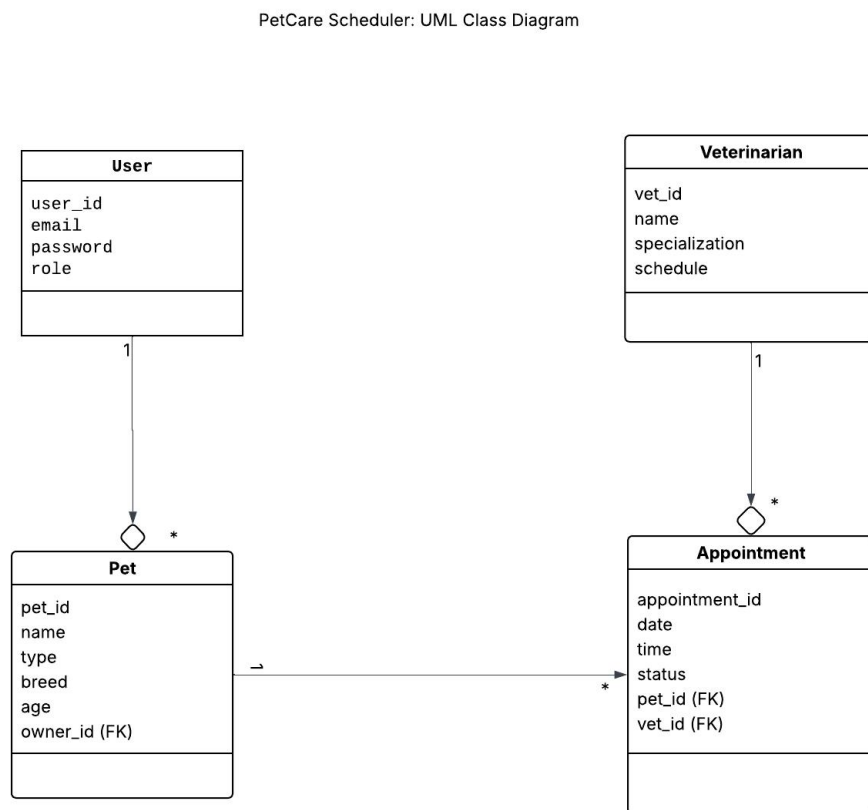


Figure 3: System UML Class Diagram

System Architecture Diagram

The architecture routes background browser fetch requests through an integrated cookie validation engine before allowing database entities to undergo edits or updates.

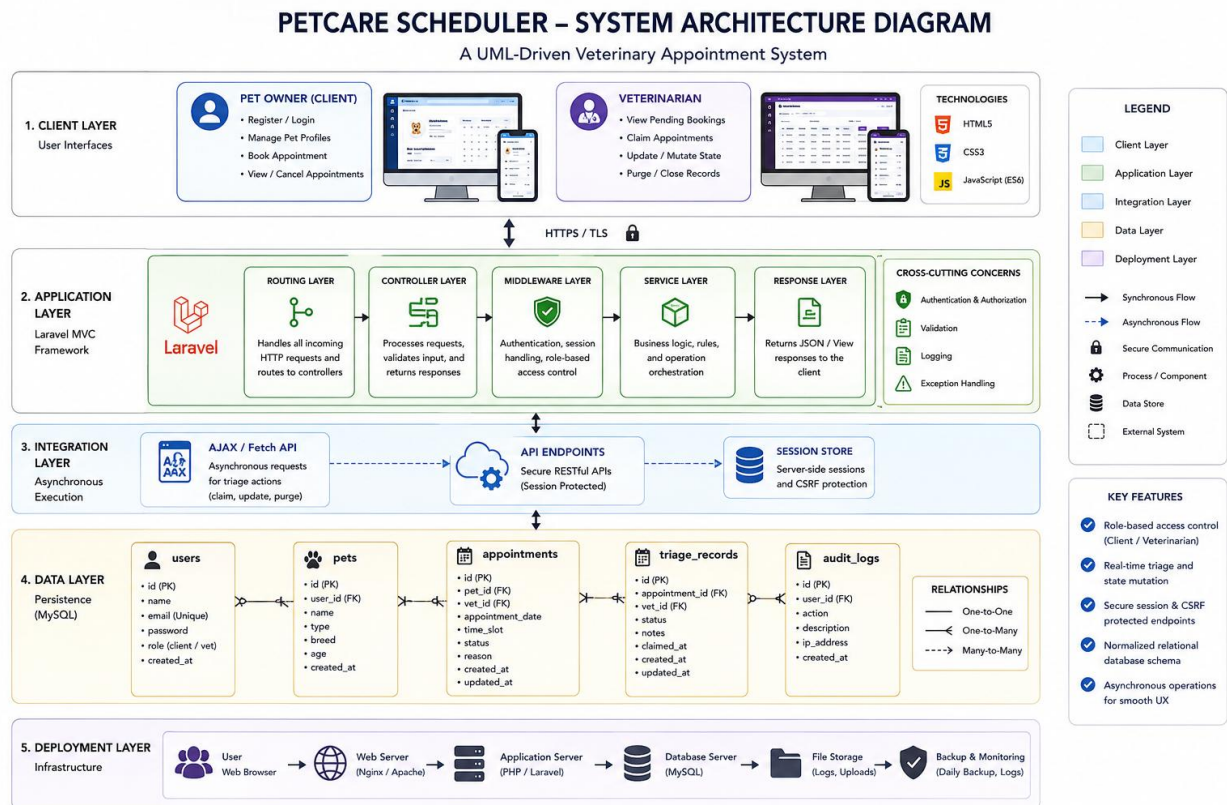


Figure 4: System Architecture Diagram

VII. Database Structure

List of Tables

1. User Table

Stores structural credentials, authorization identifiers, and role types for platform accounts.

Primary Key: user_id

Field Name	Data Type	Nullable	Key / Index	Description
user_id	BIGINT UNSIGNED	No	PK	Auto-incrementing identifier unique to each system user.
email	VARCHAR(255)	No	Unique	Unique electronic mail address used as the profile login handle.
password	VARCHAR(255)	No	None	Bcrypt encrypted string used to safeguard system entry.
role	VARCHAR(50)	No	None	Tracks user permission boundaries (client, vet).

2. Pet Table

Stores descriptive pet profile characteristics tied to their respective human owners.

Primary Key: pet_id

Foreign Key: owner_id references User(user_id) on delete cascade.

Field Name	Data Type	Nullable	Key / Index	Description
pet_id	BIGINT UNSIGNED	No	PK	Auto-incrementing identifier unique to each pet record.
name	VARCHAR(255)	No	None	Given name of the animal patient.
type	VARCHAR(100)	No	None	Animal species categorization (e.g., Canine, Feline).
breed	VARCHAR(100)	Yes	None	Specific breed profile tracking details.
age	INT	Yes	None	Recorded age value of the animal patient.
owner_id	BIGINT UNSIGNED	No	FK	Links the pet animal record back to its account owner.

3. Veterinarian Table

Stores professional specializations, names, and clinic availability frameworks for medical users.

Primary Key: vet_id

Field Name	Data Type	Nullable	Key / Index	Description
vet_id	BIGINT UNSIGNED	No	PK	Auto-incrementing identifier unique to each veterinarian.
name	VARCHAR(255)	No	None	Full professional name of the attending practitioner.
specialization	VARCHAR(255)	Yes	None	Medical focus domain (e.g., General Surgery, Dermatology).
schedule	TEXT	Yes	None	Formatted text matrix defining provider availability hours.

4. Appointment Table

Coordinates consultation records, status flows, pet associations, and assigned veterinarians.

Primary Key: appointment_id

Foreign Key 1: pet_id references Pet(pet_id) on delete cascade.

Foreign Key 2: vet_id references Veterinarian(vet_id) on delete set null.

Field Name	Data Type	Nullable	Key / Index	Description
appointment_id	BIGINT UNSIGNED	No	PK	Unique identifier mapping the appointment entry.
date	DATE	No	None	The scheduled date slot for the clinic booking.
time	TIME	No	None	The designated time block for the patient review.
status	VARCHAR(50)	No	None	State pipeline tracker (Pending, Approved, Completed, Cancelled, Rescheduled).
pet_id	BIGINT UNSIGNED	No	FK	Links the consultation record to a specific pet patient.
vet_id	BIGINT UNSIGNED	Yes	FK	Links the slot to the claimed attending provider.

VIII. Implementation and Testing

Development Process

The development of the *PetCare Scheduler* followed an iterative, UML-driven Agile methodology aligned with the provided system specifications. The process was executed in the following sequential phases:

1. **Database Schema Design and Migration:** Using the structural definitions from the UML Class Diagram, MySQL database tables (users, pets, veterinarians, and appointments) were established using backend migration files to enforce strict foreign key constraints and data types.
2. **Backend Architecture and Logic Setup:** Core object-relational models and controller mappings were programmed using PHP. These controllers handle client bookings, profile management, and veterinary triage actions.
3. **Route Configuration and Middleware Layering:** Endpoints were explicitly registered through web routing middleware to ensure that all session state histories and browser authentication data are persistently verified during interactions.
4. **Asynchronous Interface Implementation:** The frontend dashboards were built and connected via JavaScript `fetch()` web APIs, allowing real-time pipeline adjustments (such as claiming a session or approving cancellations) without requiring complete webpage reloads.

Testing Procedures Performed

To verify system stability and ensure compliance with design constraints, black-box and integration testing procedures were carried out:

- **Role-Based Access Control (RBAC) Testing:** Boundaries between "Pet Owner" and "Veterinarian" profiles were systematically tested. The application successfully restricts regular clients from accessing or modifying veterinary dashboard panels (`/vet_dashboard`).
- **Asynchronous State Transmission Verification:** Application logs and browser developer network headers were closely monitored during background operations to

verify that secure session verification data and state parameters are accurately passed to backend endpoints.

- **UI/UX Interaction and Modal Testing:** The interface was thoroughly tested to ensure user prompt overlays and operation confirmations trigger accurately during destructive actions (such as permanent record deletions).
- **Data Persistence and Relations Check:** Tests verified that creating or removing records cascades accurately through the database without leaving detached entries, perfectly following the relationships established in the UML Class Diagram (e.g., ensuring a User maps cleanly to their corresponding Pets).

Problems Encountered and Solutions Applied

1. Asynchronous Authentication Failures (HTTP 419 / Unauthenticated State)

- **Problem:** During frontend triage actions on the veterinarian dashboard, clicking update operations like "Approve Cancel & Delete" or changing statuses via the dropdown actions repeatedly failed, triggering an browser alert dialog showing an "Unauthenticated" message overlay. This error occurred because asynchronous background requests did not natively attach session context cookies, leading the backend middleware to reject the requests as unauthorized data breaches.
- **Solution:** The frontend script was reconfigured to enforce secure credential tracking. By passing the explicit cross-origin and same-origin resource properties (credentials: 'same-origin') into the JavaScript fetch() network configurations, the web application properly synchronized session details with the middleware security stack, eliminating authorization blocks.

2. Table Primary Key Lookup Inconsistencies

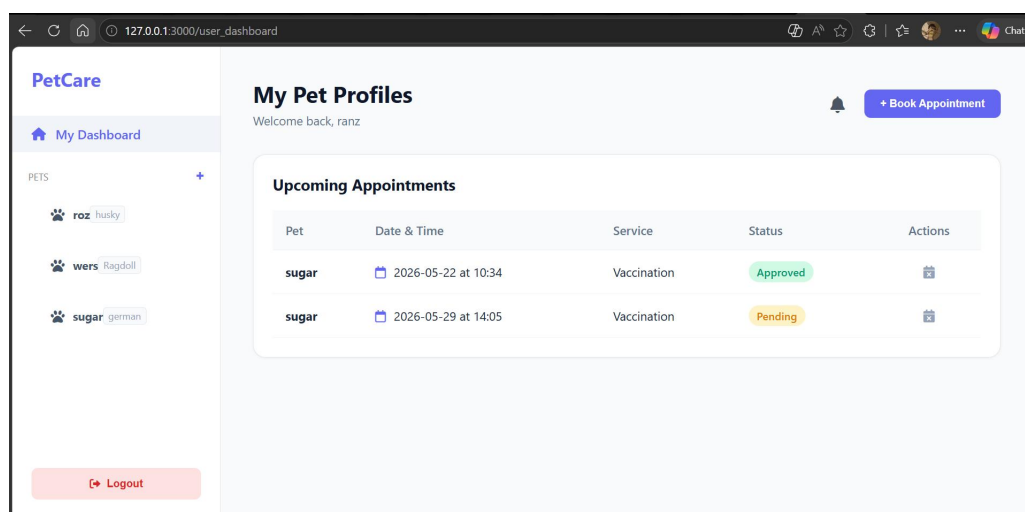
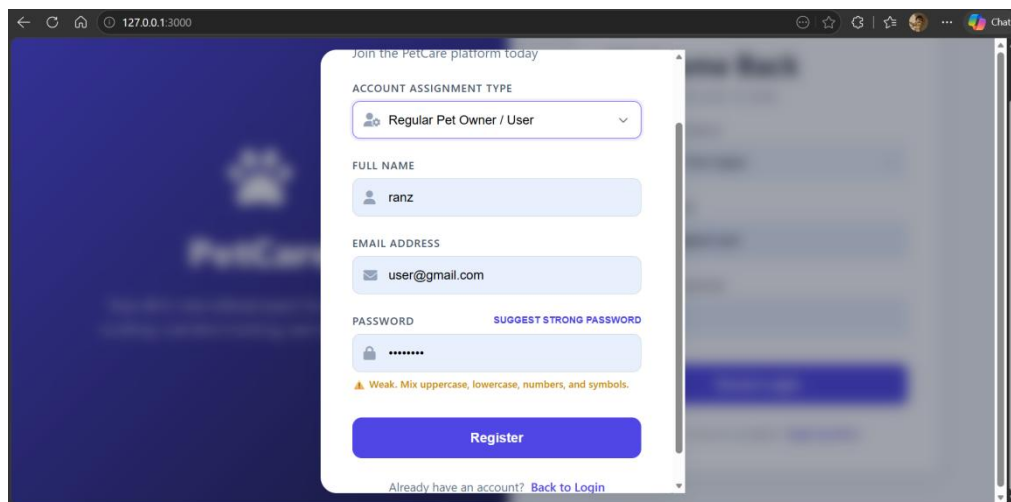
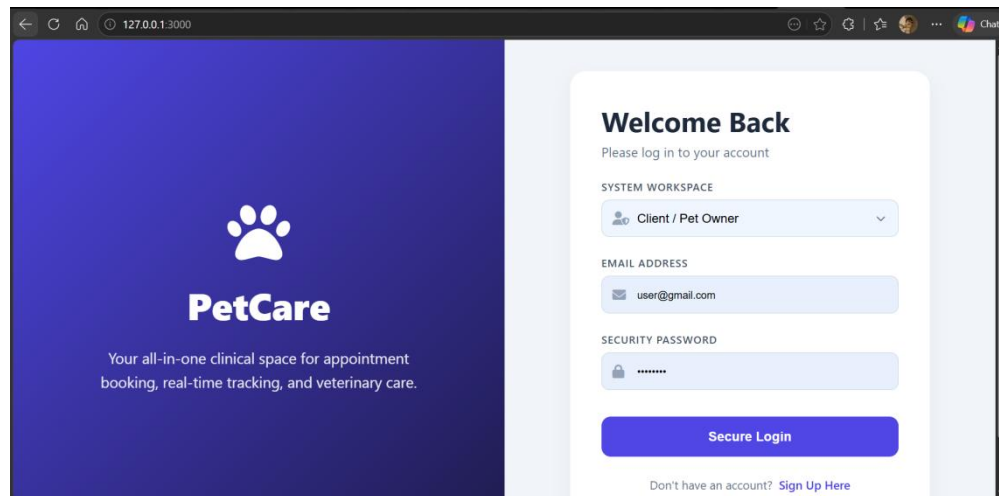
- **Problem:** Database relationship mapping queries frequently crashed with exceptions when looking up records or connecting data entries between owners, pets, and appointments. This was caused by naming mismatches within structural lookup variables, where standard framework properties conflicted with custom defined keys (user_id vs. id).

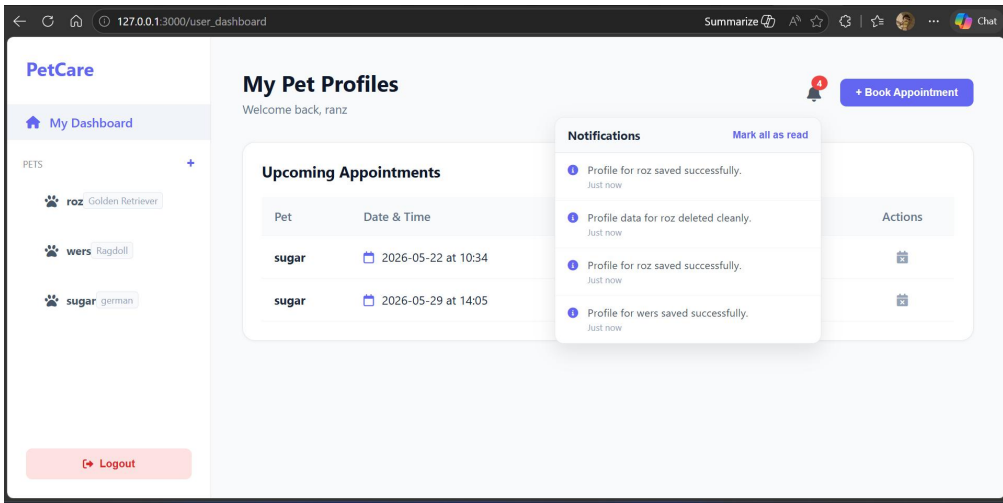
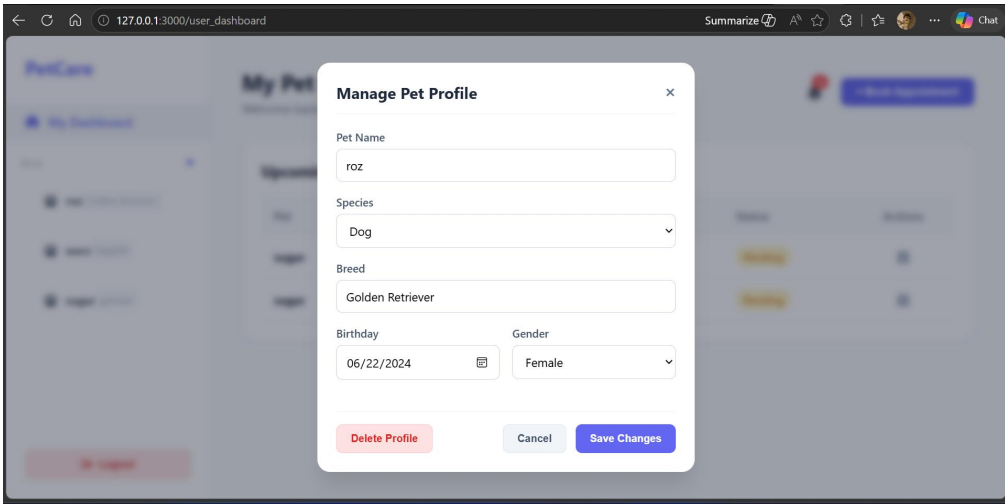
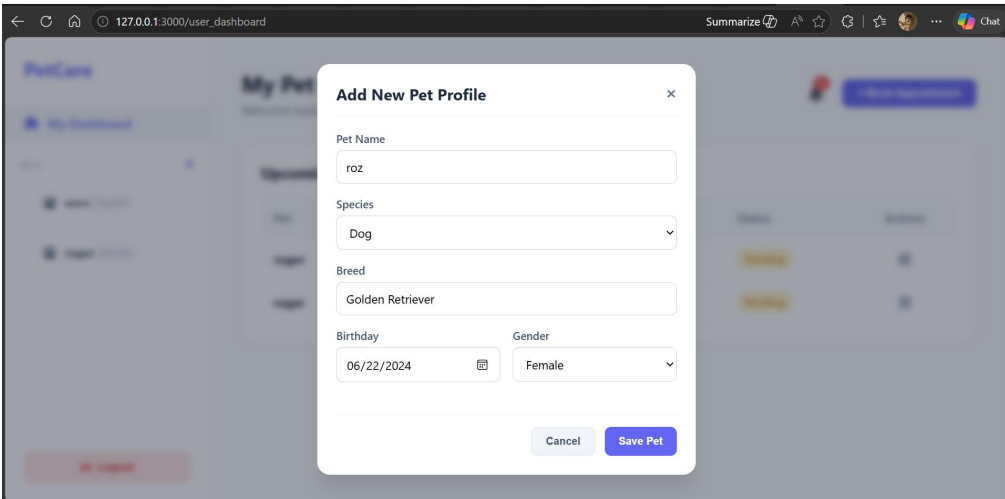
- **Solution:** Database lookup logic and Eloquent configuration definitions were normalized to support fallback primary key references (`$user->user_id ?? $user->id`), stabilizing internal data pipelines.

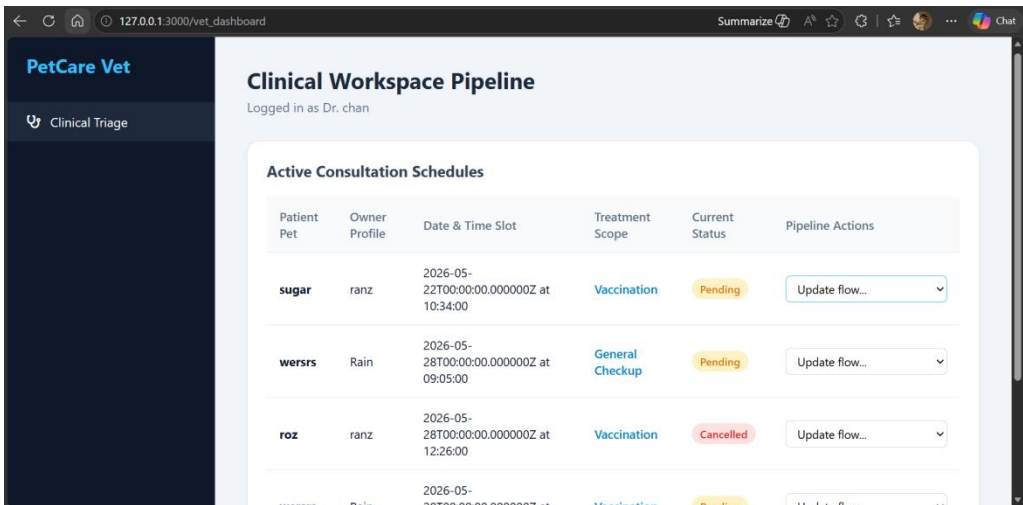
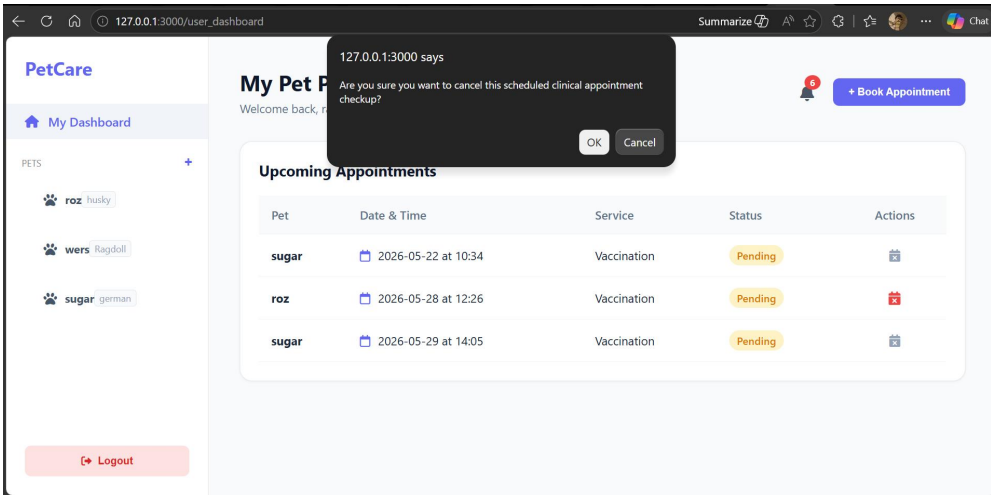
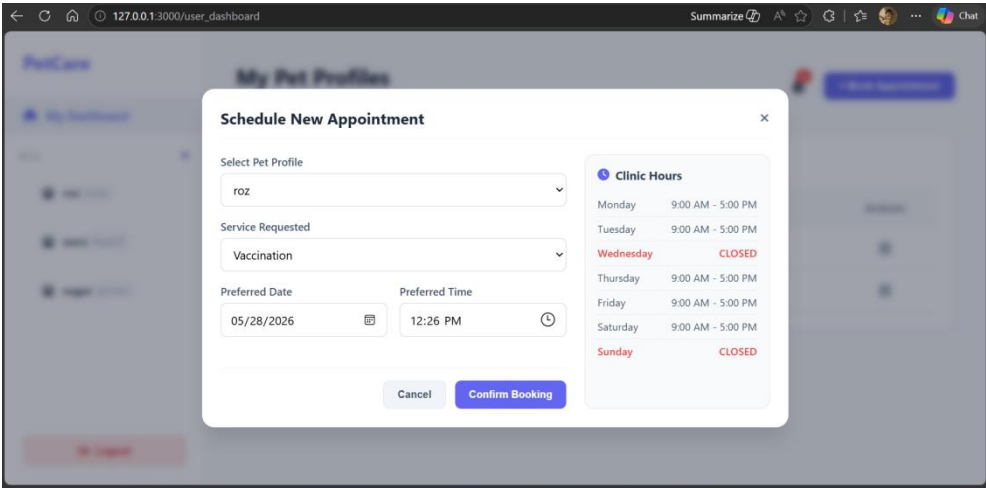
3. Caching and Stale Route Mapping Artifacts

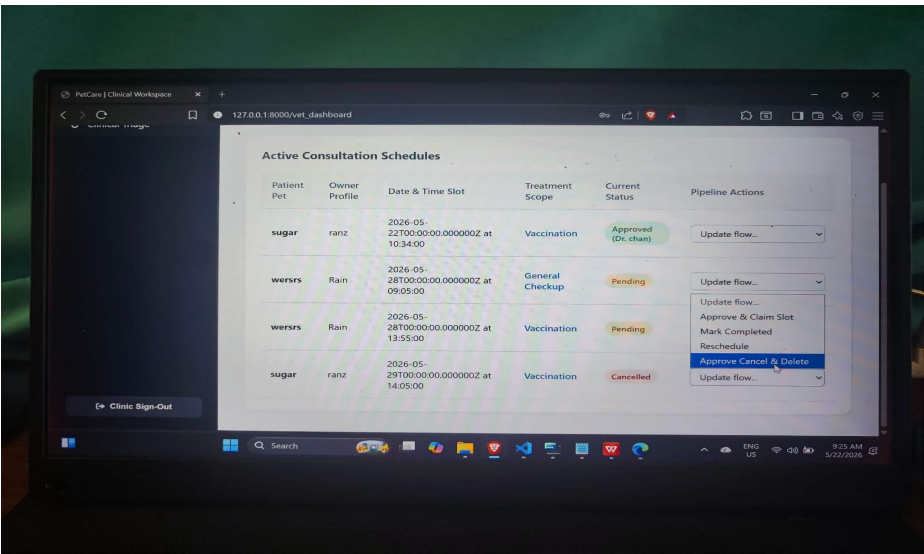
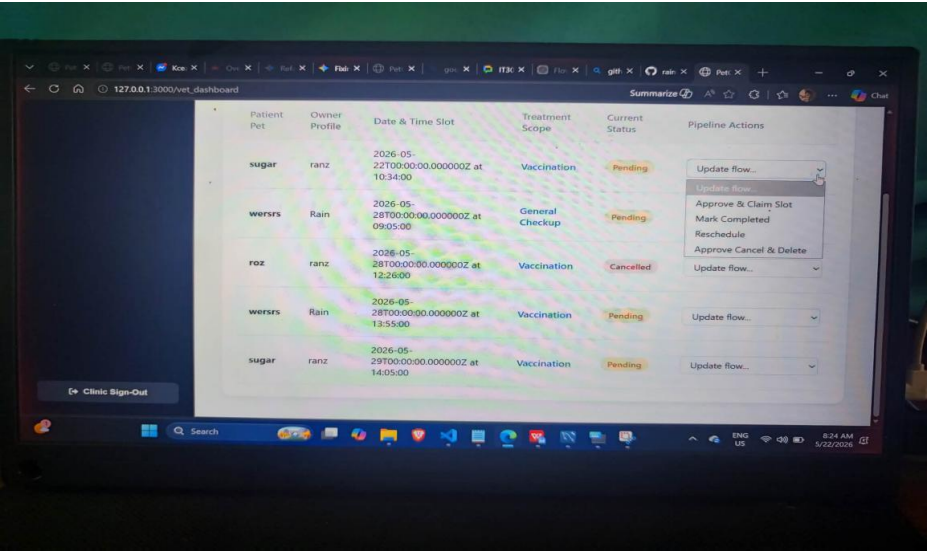
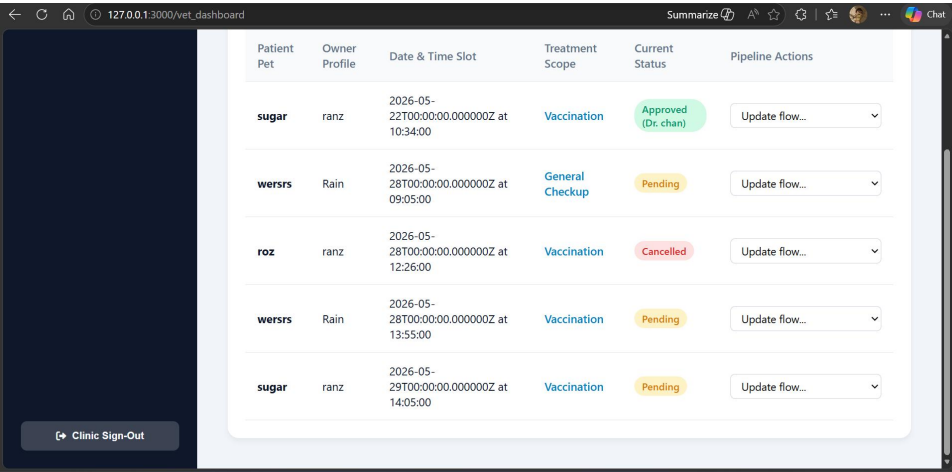
- **Problem:** After moving file pathways or updating access configurations within controller methods, the web application frequently failed to find new paths, executing outdated layout actions or displaying stale script data on the dashboard grids.
- **Solution:** To enforce changes instantly, old route definitions and framework optimization memories were systematically cleared from the development server environment using the Artisan Command-Line Interface via `php artisan route:clear`.

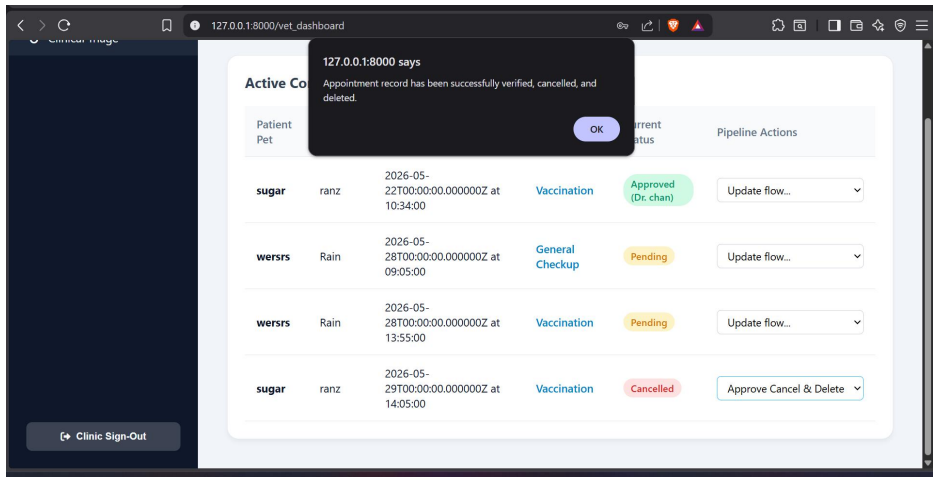
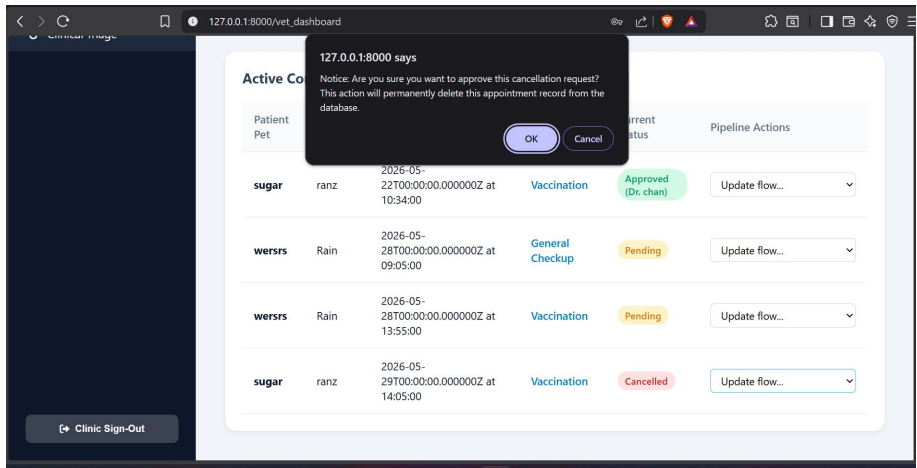
IX. Interface Design / Screenshots











X. Conclusion

Project Achievements

The *PetCare Scheduler* project provides a secure, fully functional automated system for booking appointments and managing clinical triage pipelines. Enforcing matching credential configurations across the frontend fetch layer allows the system to load appointment records, process status changes, and purge database entries smoothly without encountering application drops.

Benefits of the System

- **Organized Resource Management:** Minimizes administrative overhead and eliminates booking errors by tracking real-time status transitions.
- **Clear Patient Tracking:** Consolidates disparate pet records, client metadata, and reservation logs into an structured schema.
- **Reliable Endpoint Security:** Uses shared-origin cookie verification to block unauthenticated access requests and protect data integrity.

Learning Gained During Development

Building this system highlighted the operational differences between stateless API setups and stateful cookie routing architectures. It demonstrated the importance of keeping frontend fetch headers properly aligned with backend middleware rules. It also showed that maintaining clean database patterns and explicit foreign keys makes it much simpler to build secure, role-segmented web systems.

8. Recommendations

Future Improvements

- **Automated Conflict Flagging:** Build an automated scheduling checker that scans practitioner lists and blocks conflicting bookings before database entries are written.
- **Historical Treatment Logs:** Add an audit module to store completed appointment notes, keeping a record of past patient care directly inside the database profile.

Additional Features

- **External Client Alert Networks:** Connect the application to mail or text alert APIs to send real-time appointment confirmations and cancellation updates directly to clients' personal devices.
- **Medical Supply Tracking:** Build a companion supply module to track materials and automatically deduct clinical supplies as consultation statuses update to "Completed".

System Enhancement Ideas

- **Modular Codebase Restructuring:** Move from a monolithic framework to a modular service design to handle larger volumes of data and client requests more efficiently.
- **Interactive Health Charts:** Add data visualization tools to client dashboards, allowing pet owners to track vital pet statistics and measurement trends over time.
- **Modular Codebase Restructuring:** Move from a monolithic framework to a modular service design to handle larger volumes of data and client requests more efficiently.
- **Interactive Health Charts:** Add data visualization tools to client dashboards, allowing pet owners to track vital pet statistics and measurement trends over time.